



Dockerfile

Die quelloffene Software Docker (<https://www.ionos.de/digitalguide/server/knowhow/was-ist-docker/>) hat sich **als Standard für die Container-Virtualisierung** etabliert. Die Container-Virtualisierung setzt die Entwicklung der virtuellen Maschinen fort, jedoch mit einem bedeutenden Unterschied: anstatt ein komplettes Betriebssystem zu simulieren, wird eine einzelne Applikation in einem Container virtualisiert. Heutzutage kommen Docker-Container in allen Phasen des Software-Lifecycle, wie Entwicklung, Test und Betrieb, zum Einsatz.

Im Docker-Ökosystem existieren **verschiedene Konzepte**. Diese Bestandteile zu kennen und zu verstehen ist essenziell, um mit Docker zielführend zu arbeiten. Dazu gehören insbesondere das Docker-Image (<https://www.ionos.de/digitalguide/server/knowhow/docker-image/>), der Docker-Container (<https://www.ionos.de/digitalguide/server/knowhow/docker-container/>), sowie das Dockerfile. Wir erklären die Hintergründe und geben praktische Tipps für den Einsatz.

Inhaltsverzeichnis

1. Was ist ein Dockerfile?
2. Wie funktioniert ein Dockerfile und wie lässt sich daraus ein Image erzeugen?
3. Wie ist ein Dockerfile aufgebaut?
4. Die wichtigsten Dockerfile-Anweisungen
5. Wie lässt sich ein Dockerfile bearbeiten?

Was ist ein Dockerfile?

Das Dockerfile ist die **grundlegende Einheit im Docker-Ökosystem**. Es beschreibt die Schritte, welche zur Erzeugung eines Docker-Image führen. Der Informationsfluss folgt dabei dem zentralen Schema: Dockerfile > Docker-Image > Docker-Container

Ein Docker-Container hat eine begrenzte Lebensdauer und interagiert mit seiner Umgebung. Sie können sich einen Container **wie einen lebendigen Organismus vorstellen**. Denken Sie dabei an einen Einzeller, z. B. eine Hefezelle. Dieser Analogie folgend entspricht ein Docker-Image in etwa der Erbinformation: alle aus einem Image erzeugten Container sind gleich. Genau wie sämtliche aus einer Erbinformation geklonten Einzeller. Wie passt nun das Dockerfile ins Schema?

Ein Dockerfile definiert die Schritte zur **Erzeugung eines neuen Images**. Wichtig zu verstehen ist dabei, dass immer mit einem existierenden Basis-Image begonnen wird. Das neu erzeugte Image erbt vom Basis-Image. Hinzu kommen eine Reihe punktueller Änderungen. Auf unser Hefezellen-Beispiel bezogen entsprechen die Änderungen Mutationen. Ein Dockerfile legt für ein neues Docker-Image zwei Dinge fest:

1. Das Basis-Image, von dem das neue Image abstammt. Hiermit wird das neue Image im Stammbaum des Docker-Ökosystems verankert.
2. Eine Reihe spezifischer Änderungen, welche das neue Image vom Basis-Image unterscheiden.

Wie funktioniert ein Dockerfile und wie lässt sich daraus ein Image erzeugen?

Im Grunde genommen handelt es sich bei einem Dockerfile um eine **ganz normale Textdatei**. Das Dockerfile enthält eine Reihe von Anweisungen, welche jeweils auf einer eigenen Zeile stehen. Zur Erzeugung eines Docker-Image werden die Anweisungen hintereinander ausgeführt. Dieses Schema ist Ihnen vielleicht von der Ausführung eines Stapelverarbeitungs-Skriptes bekannt. Bei

der Ausführung werden dem Image Schritt für Schritt weitere Layer hinzugefügt. Wie das genau funktioniert, erklären wir in unserem Artikel zum Thema Docker-Image.

Digital Guide



Ein Docker-Image wird durch Ausführen der Anweisungen eines Dockerfile erzeugt. Dieser Schritt wird als **Build-Prozess** bezeichnet und wird mit Ausführen des „docker build“-Befehls gestartet. Ein zentrales Konzept ist der sogenannte Build-Context. Dieser definiert, auf welche Dateien und Verzeichnisse der Build-Prozess Zugriff hat. Dabei dient ein lokales Verzeichnis als Quelle. Der Inhalt des Quell-Verzeichnisses wird beim Aufruf von „docker build“ an den Docker-Daemon übergeben. Die in der Dockerfile enthaltenen Anweisungen erhalten Zugriff auf die im Build-Kontext enthaltenen Dateien und Verzeichnisse.

Manchmal möchte man nicht alle im lokalen Quell-Verzeichnis vorliegenden Dateien in den Build-Kontext aufnehmen. Für diese Fälle gibt es die .dockerignore-Datei. Diese dient zum **Ausschließen von Dateien und Verzeichnissen aus dem Build-Kontext**. Der Name ist angelehnt an die .gitignore-Datei von Git. Der führende Punkt im Dateinamen zeigt an, dass es sich um versteckte Datei handelt.

Wie ist ein Dockerfile aufgebaut?

Ein Dockerfile ist eine **Plain-Text-Datei mit Dateinamen „Dockerfile“**. Beachten Sie, dass dabei die Großschreibung des ersten Buchstaben zwingend vorgeschrieben ist. Die Datei enthält einen Eintrag pro Zeile. Wir zeigen hier den generellen Aufbau eines Dockerfile:

```
1 | # Kommentar
2 | ANWEISUNG Argumente
```

Neben Kommentaren enthält das Dockerfile **Anweisungen und Argumente**. Diese beschreiben den Aufbau des Images.

Kommentare und Parser-Anweisungen

Kommentare enthalten primär für den Menschen gedachte Informationen. Wie beispielsweise, aus den Programmiersprachen Python, Perl und Ruby bekannt, beginnen Kommentare in einem Dockerfile mit einem **Rautezeichen (#)**. Während des Build-Prozesses werden Kommentar-Zeilen vor der weiteren Verarbeitung entfernt. Dabei gilt zu beachten, dass nur Zeilen, welche mit dem Raute-Zeichen beginnen als Kommentarzeilen erkannt werden.

Hier steht ein gültiger Kommentar:

```
1 | # unser Base-Image
2 | FROM busybox
```

Hingegen wird hier ein Fehler erzeugt, da das Rautezeichen nicht am Anfang der Zeile steht:

```
1 | FROM busybox # unser Base-Image
```

Als Variante der Kommentare gibt es die **Parser-Anweisungen**. Diese sind in Kommentar-Zeilen enthalten und müssen am Anfang des Dockerfile stehen. Ansonsten werden sie als Kommentare behandelt und beim Build entfernt. Ferner gilt zu beachten, dass eine bestimmte Parser-Anweisung **nur ein einziges Mal pro Dockerfile** verwendet werden kann.

Zum Zeitpunkt der Artikelerstellung existieren lediglich zwei Arten von Parser-Anweisungen: **„syntax“ und „escape“**. Die Parser-Anweisung „escape“ definiert das zum Einsatz kommende Escape-Symbol. Dieses wird genutzt, um Anweisung über mehrere Zeilen zu schreiben, sowie Sonderzeichen auszudrücken. Die „syntax“-Parser-Anweisung legt fest, nach welchen Regeln der Parser die Dockerfile-Anweisungen zu verarbeiten hat. Hier ein Beispiel:

```
1 | # syntax=docker/dockerfile:Digital Guide
2 | # escape=\\
```



(<https://www.ionos.de/digitalguide/>)

Anweisungen, Argumente und Variablen

Die Anweisungen bilden den Hauptteil des Dockerfile-Inhalts. Nacheinander ausgeführt **beschreiben die Anweisungen den spezifischen Aufbau eines Docker-Image**. Ähnlich wie Befehle auf der Kommandozeile nehmen die Anweisungen Argumente entgegen. Manche Anweisungen sind direkt mit spezifischen Kommandozeilen-Befehlen vergleichbar. So existiert eine COPY-Anweisung, welche Dateien und Verzeichnisse kopiert und in etwa dem cp -Befehl (<https://www.ionos.de/digitalguide/server/konfiguration/linux-cp-befehl/>) auf der Kommandozeile entspricht. Anders als auf der Kommandozeile gibt es für manche der Dockerfile-Anweisungen spezifische Regeln für deren Abfolge. Ferner können bestimmte Anweisungen nur einmal pro Dockerfile auftreten.

! Hinweis

Die Großschreibung der Anweisungen ist nicht zwingend vorgeschrieben. Sie sollten der Konvention dennoch folgen, wenn Sie ein Dockerfile erstellen.

Bei den Argumenten gilt es zu **unterscheiden zwischen fest-kodierten und variablen Teilen**. Der „12-factor App“ (<https://12factor.net/de/>)-Methodologie folgend, verwendet Docker Umgebungsvariablen zur Konfiguration der Container. Innerhalb einer Dockerfile werden Umgebungsvariablen mit der ENV-Anweisung definiert. So weist man der Umgebungsvariable einen Wert zu.

Die in Umgebungsvariablen gespeicherten Werte lassen sich auslesen und als **variable Teile von Argumenten** nutzen. Dazu kommt eine spezielle Syntax zum Einsatz, welche an Shell-Skripte (<https://www.ionos.de/digitalguide/server/knowhow/wie-man-ein-einfaches-shell-skript-schreibt/>) erinnert. Der Name der Umgebungsvariable wird mit einem Dollarzeichen versehen: \$env_var. Ferner existiert eine alternative Schreibweise zum expliziten Begrenzen des Variablennamens. Hierbei wird der Variablenname in geschweifte Klammern eingebettet: \${env_var}. Schauen wir uns ein konkretes Beispiel an:

```
1 | # Variable 'user' auf Wert 'admin' setzen
2 | ENV user="admin"
3 | # Nutzernamen auf 'admin_user' setzen
4 | USER ${user}_user
```

Die wichtigsten Dockerfile-Anweisungen

Wir stellen hier die wichtigsten Dockerfile-Anweisungen vor. Traditionell durften manche Anweisungen — insbesondere FROM — nur einmal pro Dockerfile auftreten. Mittlerweile gibt es Multi-Stage Builds (<https://docs.docker.com/develop/develop-images/multistage-build/#use-multi-stage-builds>). Diese beschreiben mehrere Images in einer Dockerfile. Die Einschränkung bezieht sich dann auf jede einzelne Build-Stage.

Anweisung	Beschreibung	Kommentar
FROM	Basis-Image festlegen	Muss als erste Anweisung auftreten; nur ein Eintrag pro Build-Stage
ENV	Umgebungsvariablen für Build-Prozess und Container-Laufzeit setzen	—

ARG	Kommandozeilenparameter für Build FROM-Anweisungen deklarieren (https://www.ionos.de/digitalguide/)	Darf vor FROM-Anweisung auftreten
WORKDIR	Aktuelles Verzeichnis wechseln	–
USER	Nutzer und Gruppenzugehörigkeit wechseln	–
COPY	Dateien und Verzeichnisse in das Image kopieren	Legt neuen Layer an
ADD	Dateien und Verzeichnisse in das Image kopieren	Legt neuen Layer an; von Nutzung wird abgeraten
RUN	Befehl im Image während des Build-Prozesses ausführen	Legt neuen Layer an
CMD	Standard-Argumente für Container-Start festlegen	Nur ein Eintrag pro Build-Stage
ENTRYPOINT	Standard-Befehl für Container-Start festlegen	Nur ein Eintrag pro Build-Stage
EXPOSE	Port-Zuweisungen für laufenden Container definieren	Ports müssen beim Starten des Containers aktiv geschaltet werden
VOLUME	Verzeichnis im Image beim Start des Containers im Host-System als Volumen einbinden	–



FROM-Anweisung

Die From-Anweisung **legt das Basis-Image fest**, auf dem die nachfolgenden Anweisungen operieren. Diese Anweisung darf pro Build-Stage nur einmal vorhanden sein und muss als erste Anweisung auftreten. Dabei gibt es eine Einschränkung: die ARG-Anweisung darf vor der FROM-Anweisung auftreten. So ist es möglich, beim Starten des Build-Prozesses per Kommandozeilenargument festzulegen, welches Image genau als Basis-Image verwendet wird.

Jedem Docker-Image muss ein Basis-Image zugrunde liegen. Anders ausgedrückt: **jedes Docker-Image hat genau ein Vorgänger-Image**. Es ergibt sich ein klassisches „Henne-Ei-Problem“: Irgendwo muss die Abstammungskette ihren Anfang haben. Im Docker-Universum beginnt die Abstammung mit dem „scratch“-Image (https://hub.docker.com/_/scratch/). Dieses minimale Image dient als Ursprung eines jeden Docker-Images.

! Hinweis

Auf Englisch bedeutet „from scratch“, dass etwas aus Grundzutaten hergestellt wird. Der Begriff findet beim Backen und Kochen Verwendung. Beginnt ein Dockerfile mit der Zeile „FROM scratch“ wird also darauf angespielt, dass das Image von Grund auf neu zusammengestellt wird.

ENV- und ARG-Anweisungen Digital Guide

Diese beiden Anweisungen **weisen einer Variable einen Wert zu**. Die Unterscheidung zwischen beiden Anweisungen liegt primär darin, woher die Werte stammen und in welchem Kontext die Variablen verfügbar sind. Betrachten wir zunächst die ARG-Anweisung. [\(https://www.ionos.de/digitalguide/\)](https://www.ionos.de/digitalguide/)

Mit der **ARG-Anweisung** wird innerhalb des Dockerfile eine Variable deklariert, welche **ausschließlich für die Dauer des Build-Prozesses verfügbar** ist. Der Wert einer mit ARG deklarierten Variable wird beim Starten des Build-Prozesses als Kommandozeilen-Argument übergeben. Hier ein Beispiel; wir deklarieren die Build-Variable „user“:

```
1 | ARG user
```

Beim Starten des Build-Prozesses übergeben wir den eigentlichen Wert der Variable:

```
1 | docker build --build-arg user=admin
```

Bei der Deklaration der Variable **kann optional ein Default-Wert festgelegt werden**. Wird beim Starten des Build-Prozesses kein passendes Argument übergeben, wird die Variable mit dem Default-Wert versehen:

```
1 | ARG user=tester
```

Ohne Nutzung von „--build-arg“ enthält die Variable „user“ den Default-Wert „tester“:

```
1 | docker build
```

Mithilfe der **ENV-Anweisung** definieren wir eine Umgebungsvariable. Im Gegensatz zur ARG-Anweisung existiert eine mit ENV definierte Variable **sowohl während des Build-Prozesses als auch während der Container-Laufzeit**. Für die ENV-Anweisung gibt es zwei erlaubte Schreibweisen.

1. Empfohlene Schreibweise:

```
1 | ENV version="1.0"
```

2. Alternative Schreibweise, für Rückwärts-Kompatibilität:

```
1 | ENV version 1.0
```

💡 Tipp

Die Funktionalität der ENV-Anweisung entspricht grob der des „export“-Befehls auf der Kommandozeile.

WORKDIR- und USER-Anweisungen

Die **WORKDIR-Anweisung** dient zum **Wechseln der Verzeichnisse** während des Build-Prozesses, sowie beim Starten des Containers. Der Aufruf von WORKDIR gilt für alle nachfolgenden Anweisungen. Während des Build-Prozesses werden RUN-, COPY- und ADD-Anweisungen beeinflusst; während der Container-Laufzeit CMD- und ENTRYPOINT-Anweisungen.



Digital Guide



Die WORKDIR-Anweisung entspricht in etwa dem Befehl `cd` auf der Kommandozeile.

Analog zum Wechseln des Verzeichnisses erlaubt die **USER-Anweisung** das **Wechseln des aktuellen (Linux)-Nutzers**. Optional lässt sich die Gruppenzugehörigkeit des Nutzers festlegen. Der Aufruf von USER gilt für alle nachfolgenden Anweisungen. Während des Build-Prozesses werden RUN-Anweisungen durch Nutzer und Gruppenzugehörigkeit beeinflusst; während der Container-Laufzeit gilt dies für CMD- und ENTRYPOINT-Anweisungen.



Die USER-Anweisung entspricht in etwa dem Befehl `su` auf der Kommandozeile.

COPY- und ADD-Anweisungen

Die COPY- und ADD-Anweisungen dienen beide dazu, dem Docker-Image **Dateien und Verzeichnisse hinzuzufügen**. Beide Anweisungen erzeugen einen neuen Layer, welcher auf das bestehende Image gestapelt wird. Bei der COPY-Anweisung ist die Quelle immer der Build-Context. Im folgenden Beispiel kopieren wir eine Readme-Datei (<https://www.ionos.de/digitalguide/websites/web-entwicklung/readme-datei/>) vom Unterverzeichnis „doc“ im Build-Context in das Top-Level-Verzeichnis „app“ des Images:

```
1 | COPY ./doc/readme.md /app/
```



Die COPY-Anweisung entspricht in etwa dem Befehl `cp` auf der Kommandozeile.

Die ADD-Anweisung verhält sich größtenteils identisch, kann jedoch **URL-Ressourcen außerhalb des Build-Kontexts** abrufen und entpackt komprimierte Dateien. In der Praxis führt dies unter Umständen zu unerwarteten Nebeneffekten. Daher wird ausdrücklich vom Gebrauch der ADD-Anweisung abgeraten. Sie sollten in den meisten Fällen ausschließlich die COPY-Anweisung verwenden.

RUN-Anweisung

Bei der RUN-Anweisung handelt es sich um eine der am häufigsten auftretenden Dockerfile-Anweisungen. Mit der RUN-Anweisung weisen wir Docker an, **während des Build-Prozesses einen Kommandozeilen-Befehl auszuführen**. Die dabei entstehenden Änderungen werden als neuer Layer auf das bestehende Image gestapelt. Für die RUN-Anweisung existieren zwei Schreibweisen:

1. **„Shell“-Schreibweise:** Die an RUN übergebenen Argumente werden in der Standard-Shell des Images ausgeführt. Dabei werden spezielle Symbole und Umgebungsvariablen den Shell-Regeln folgend ersetzt. Hier ein Beispiel für einen Aufruf, welcher den aktuellen Nutzer begrüßt und dabei eine Subshell „\$()“ verwendet:

```
1 | RUN echo "Hello $(whoami)"
```

2. **„Exec“-Schreibweise:** Anstatt einen Befehl an die Shell zu übergeben, wird eine ausführbare Datei direkt aufgerufen. Dabei werden ggf. weitere Argumente übergeben. Hier ein Beispiel für einen Aufruf, welcher das Dev-Tool „npm“ aufruft und anweist, das Skript „build“ auszuführen:

```
1 | CMD ["npm", "run", " build"] Digital Guide
```



(<https://www.ionos.de/digitalguide/>)

! Hinweis

Prinzipiell lassen sich mit der RUN-Anweisung manche der anderen Docker-Anweisungen ersetzen. Beispielsweise ist der Aufruf „RUN cd src“ ungefähr gleichwertig zu „WORKDIR src“. Jedoch erzeugt dieser Ansatz Dockerfiles, welche sich bei wachsendem Umfang schlechter Lesen und Verwalten lassen. Sie sollten daher nach Möglichkeit spezialisierte Anweisungen nutzen.

CMD- und ENTRYPOINT-Anweisungen

Die RUN-Anweisung führt einen Befehl während des Build-Prozesses aus und **legt dabei einen neuen Layer im Docker-Image an**. Demgegenüber führt die CMD- bzw. ENTRYPOINT-Anweisung einen Befehl beim Starten des Containers aus. Der Unterschied zwischen den beiden Anweisungen ist subtil:

- ENTRYPOINT wird genutzt, um einen Container zu erzeugen, welcher **beim Starten immer dieselbe Aktion ausführt**. Der Container verhält sich also wie eine ausführbare Datei.
- CMD wird genutzt, um einen Container zu erzeugen, welcher **beim Starten ohne weitere Parameter eine definierte Aktion ausführt**. Die voreingestellte Aktion lässt sich durch geeignete Parameter leicht überschreiben.

Gemeinsam haben beide Anweisungen, dass Sie **nur ein einziges Mal pro Dockerfile auftreten dürfen**. Es ist jedoch möglich, beide Anweisungen zu kombinieren. In diesem Fall definiert ENTRYPOINT die auszuführende Standard-Aktion beim Starten des Containers, während CMD leicht zu überschreibende Parameter für die Aktion definiert.

Unser Dockerfile-Eintrag:

```
1 | ENTRYPOINT ["echo", "Hello"]
2 | CMD ["World"]
```

Die dazugehörigen Befehle auf der Kommandozeile:

```
1 | # Ausgabe "Hello World"
2 | docker run my_image
3 | # Ausgabe "Hello Moon"
4 | docker run my_image Moon
```

EXPOSE-Anweisung

Docker-Container kommunizieren über das Netzwerk. Im Container laufende Services werden über festgelegte Ports angesprochen. Die EXPOSE-Anweisung **dokumentiert Port-Zuweisungen** und unterstützt die Protokolle TCP und UDP. Wird ein Container mit „docker run -P“ gestartet, lauscht der Container auf den per EXPOSE definierten Ports. Alternativ lassen sich die zugewiesenen Ports mit „docker run -p“ überschreiben.

Hier ein Beispiel. Unser Dockerfile enthält die folgenden EXPOSE-Anweisungen:

```
1 | EXPOSE 80/tcp
2 | EXPOSE 80/udp
```

Dann bieten sich die folgenden Wege, die Ports beim Starten des Containers aktiv zu schalten:

```
1 # Container lauscht für TCP-Traffic auf Port 80
2 docker run -P
3 # Container lauscht für TCP-Traffic auf Port 81
4 docker run -p 81:81/tcp
```

Digital Guide

(https://www.ionos.de/digitalguide/)



VOLUME-Anweisung

Ein Dockerfile definiert ein Docker-Image, welches aus übereinander gestapelten Layers besteht. Die Layers sind schreibgeschützt, so dass beim Starten eines Containers immer derselbe Zustand garantiert ist. Wir benötigen einen Mechanismus, um Daten zwischen dem laufenden Container und dem Host-System auszutauschen. Die VOLUME-Anweisung **definiert einen „Mount Point“ innerhalb des Containers**.

Betrachten wir den folgenden Ausschnitt eines Dockerfile. Wir legen ein Verzeichnis „shared“ im Top-Level-Verzeichnis des Images an. Ferner legen wir fest, dass dieses Verzeichnis beim Start des Containers im Host-System eingebunden wird:

```
1 RUN mkdir /shared
2 VOLUME /shared
```

Beachten Sie, dass wir innerhalb der Dockerfile nicht den eigentlichen Pfad auf dem Host-System festlegen können. Standardmäßig werden per VOLUME-Anweisung definierte Verzeichnisse auf dem Host-System unterhalb von „/var/lib/docker/volumes/“ eingebunden.

Wie lässt sich ein Dockerfile bearbeiten?

Zur Erinnerung: Bei einem Dockerfile handelt es sich um eine (Plain)-Text-Datei. Diese lässt sich mit gängigen Methoden bearbeiten; am häufigsten kommt wohl ein **Plain-Text-Editor** zum Einsatz. Dabei kann es sich um Editor mit grafischer Benutzeroberfläche handeln. An Optionen mangelt es hier nicht; zu den beliebtesten Editoren zählen VSCode, Sublime Text (https://www.ionos.de/digitalguide/websites/web-entwicklung/sublime-text-editor-im-ueberblick/), Atom und Notepad++. Alternativ stehen auf der Kommandozeile eine Reihe von Editoren zur Verfügung. Neben den Urgesteinen Vim bzw. Vi sind die minimalen Editoren Pico und Nano weit verbreitet.

! Hinweis

Sie sollten eine Plain-Text-Datei ausschließlich mit dafür geeigneten Editoren bearbeiten. Keinesfalls sollten Sie zum Bearbeiten eines Dockerfile einen Word-Prozessor wie Microsoft Word, Apple Pages oder Libre- bzw. OpenOffice verwenden.

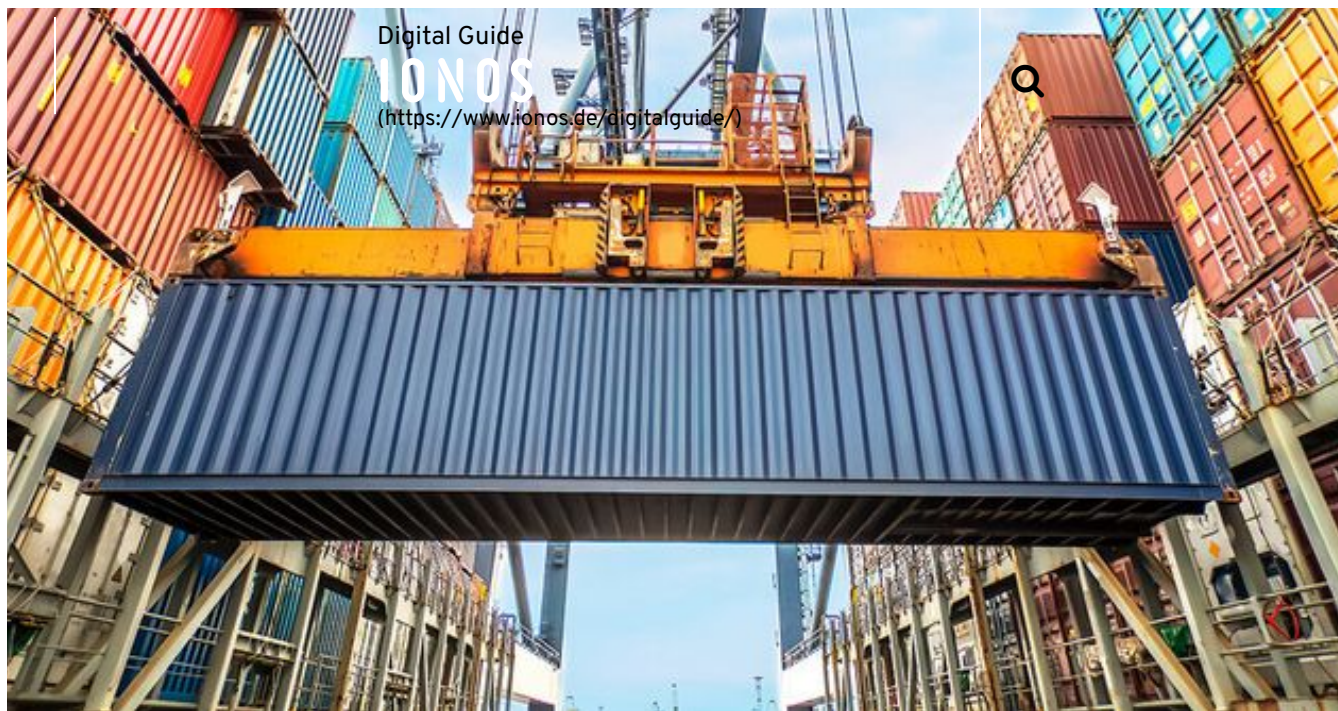
© 06.07.22 | KnowHow (https://www.ionos.de/digitalguide/server/knowhow/)

AKL .CC .CC .CC .CC
'stt' 'stt' 'stt' 'stt'

Cloud (https://www.ionos.de/digitalguide/tags/cloud/)

Ähnliche Artikel





Digital Guide
IONOS
(<https://www.ionos.de/digitalguide/>)



Das Docker-Ökosystem: Die beliebtesten Docker-Tools im Überblick

© 24.09.2020 | KnowHow

Docker hat den Einsatz von Software-Containern mit Bemühungen zur Standardisierung deutlich vereinfacht. Die schlanke Container-Plattform ist Teil eines stetig wachsenden Ökosystems, das Anwendern umfangreiche Anwendungsmöglichkeiten im Rahmen des Software-Deployments, der Orchestrierung von Containern und Multi-Container-Anwendungen sowie der Continuous Integration (CI) und Continuous Delivery...

taig attan ita de/c

Passende Produkte



IONOS Cloud

[Zu den Tarifen ►](#)

Domain Bundle Aktion

Nur für kurze Zeit: So erhalten Sie zwei Gratis-Domains!

Kaufen Sie eine .de-Domain und erhalten Sie eine .com und .info Domain gratis dazu.

[Domain kaufen ►](#)



Beliebte Artikel



Über IONOS (<https://www.ionos.de/unternehmen>) Jobs (<https://jobs.ionos.de/karriere/wer-wir-sind>)
Newsroom (<https://www.ionos.de/newsroom/>) Hilfe-Center (<https://www.ionos.de/hilfe>)
Startup Guide (<https://www.ionos.de/startupguide/>) AGB (<https://www.ionos.de/terms-gtc/terms>)
Datenschutz (<https://www.ionos.de/terms-gtc/terms-privacy>) Impressum (<https://www.ionos.de/impressum>)
© 2023 IONOS SE (<https://www.ionos.de/>)